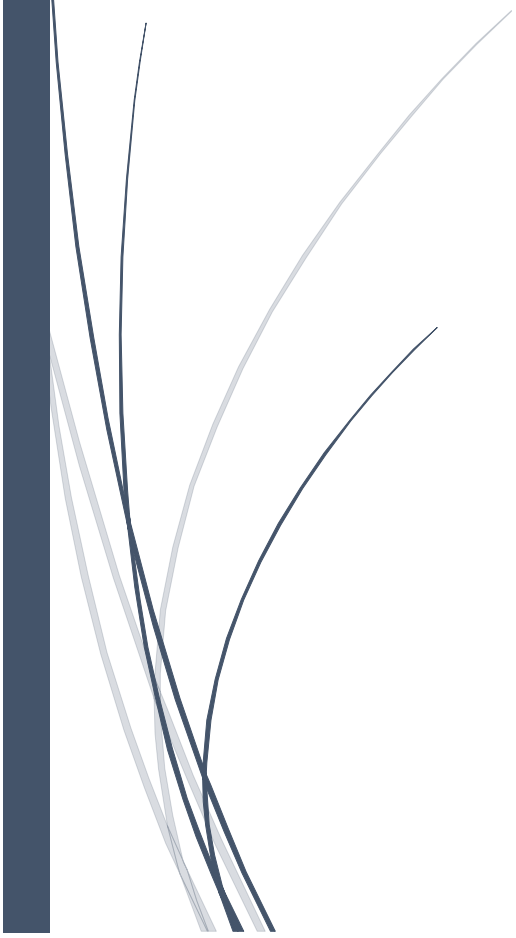
@VANZA

UT1. Introducción al Lenguaje Python

CFGS

Desarrollo de Aplicaciones Multiplataforma



Dpto. Informática
IES AUGUSTÓBRIGA (NAVALMORAL DE LA MATA)

Contenido

1. Introducción. 2
2. Evolución de Python 2
3. Características de Python 3
4. Ámbitos de uso. 5
4.2. Programación a nivel de sistema operativo. 5
4.3. Aplicaciones con Interfaz de usuario. 6
4.4. Aplicaciones web e iteracción con servicios web. 6
4.5. Interacción con servicios de Internet. 6
4.6. Gestión de contenido. 7
4.7. Aplicaciones científicas y manejo de datos. 7
4.8. Inteligencia artificial y Python 9
5. Guía de estilos en Python (PEP-8) 9
6. Distribuciones Python. 12
7. El intérprete de Python 14
7.1. Estructura del intérprete. 14
8. Implementaciones de Python 15
8.1. Desarrollar programas en Python. 16
9. Primeros programas en Python con VSC 17

1. Introducción.

En el año 1989, un joven holandés de 24 años llamado **Guido Van Rossum,** con al idea de mejorar la interfaz de usuario que utilizaban en su empresa (CWI) sobre el sistema operativo Amoeba, desarrolló el lenguaje de programación Python. Algo que iba a ser un proyecto pequeño, se convirtió en el germen de lo que hoy conocemos como uno de los mejores lenguajes de programación (si no el mejor).

El nombre del lenguaje no es debido a ninguna serpiente sino a la afición que tenía Van Rossum por una serie de televisión de los **Monty Python** (El Circo Volador).

Van Rossum ha sido el principal autor del lenguaje a lo largo de su evolución, dirigiendo su rumbo hasta que, en julio de 2018, abandonó su cargo de BDL (algo así como “el dictador benevolente de por vida”) dejándolo en manos de una comisión directiva de cinco personas de la que él también forma parte.

2. Evolución de Python

A lo largo de los años, Python ha sufrido numerosos cambios. Éstos se gestionan mediante lo que se conoce como PEP (Propuestas de Mejora de Python, de su traducción del inglés - Python Enhancement Proposals –.

Estas PEP son propuestas que se reciben de la comunidad Python, muy numerosa a nivel global y que, una vez aprobadas (o no) se integran en el código del lenguaje, la documentación o la parte específica de que se trate.

Las principales versiones con los cambios más destacados y sus fechas de lanzamientos son las siguientes:

VERSIÓN	LANZAMIENTO	DESCRIPCIÓN
0.9	Febrero 1991	Primera versión publicada por Van Rossum. Incorporaba listas, diccionarios, clases, herencia, cadenas de caracteres y otras muchas características que hacían prever su potencial.
1.4	Octubre 1996	Añade los parámetros por clave-valor y los números complejos.
2.1	Diciembre 2001	Se modifica el código para soportar ámbitos anidados y ámbitos estáticos. Se modifica la licencia a Python Software Fundation License. La fundación se crea como una organización sin ánimo de lucro,

		organizadora y dueña del código, documentación y especificaciones.
2.7	Julio 2010	Es la última versión de la rama 2.X. En 2014 se anuncia que ésta será la última versión de 2.X y se invita "activamente" a los usuarios a migrar a la nueva versión 3.
3.0	Diciembre 2008	Se lanza al mismo tiempo que la versión 2.6. Se la conoce como la versión Python 3000. Se hacen cambios en la parte principal del lenguaje, eliminando redundancia de código e introduciendo grandes incompatibilidades con la versión 2.
3.9	Octubre 2020	Se añade el paquete *zoneinfo* para facilitar el uso de zonas horarias en fechas, se añade el operador de unión (|) para diccionarios y muchas características más del lenguaje. Se han borrado muchas funciones presentes para compatibilidad con la versión 2, lo que anuncia que se irá dejando de lado el uso de la antigua versión.
3.12	Octubre 2023	La última versión estable del lenguaje Python. Se produce una combinación de cambios en el lenguaje y la biblioteca estándar. En ésta, se centran en limpiar las API obsoletas, su usabilidad y su corrección.

En el siguiente enlace podrás encontrar información actualizada sobre las versiones más recientes de Python.

Versiones Python

3. Características de Python

Estas son las características principales de Python:

- Es un lenguaje de Alto Nivel.
- De propósito general.
- Multiparadigma (principalmente imperativo, orientado a objetos y funcional).
- De tipado dinámico
- Fuertemente tipado.

El lenguaje Python tiene otros aspectos importantes que debemos tener en cuenta.

La gramática de Python hace que sea un lenguaje con una sintaxis muy sencilla, simple y clara, que permite desarrollar programas de forma muy intuitiva. Esto hace que leer los programas Python resulte sencillo y sea un lenguaje que nos permite aprender, entender y recordar de una forma más amable y eficiente.

Algunos aspectos importantes a destacar son:

- Los bloques lógicos se definen utilizando indentación en lugar de caracteres de apertura y cierre de bloque como “{}” en Java, por ejemplo.
- Las expresiones simples no necesitan el uso de paréntesis
- Para la separación de instrucciones no se necesita el comúnmente utilizado carácter “;” y se utiliza simplemente el salto de línea. (Aunque permite utilizar el “;”)
- Su sistema de recolección de basura permite al desarrollador despreocuparse de la gestión de memoria.

Uno de los principios de Python es que los simple es siempre mejor que lo complejo.

Mira el siguiente ejemplo de código Python y observa como la función no utiliza las llaves de apertura y cierre para recoger el código y sí utiliza indentación para definir su bloque lógico. Además, se aprecia que ninguna instrucción finaliza con “;”.

Observa igualmente que la expresión condicional **if numero > 0** no necesita paréntesis.

```
# Definición de una función simple que suma los números de una lista
def suma_lista(lista):
    total = 0  # Inicialización de la variable total
    for numero in lista:  # Recorre cada número en la lista
        if numero > 0:  # Condición simple sin necesidad de paréntesis
            total += numero  # Añade el número al total si es mayor que 0
    return total  # Devuelve el total calculado

# Uso de la función con una lista de ejemplo
resultado = suma_lista([3, -1, 4, 0, 5])
print("El resultado de la suma es:", resultado)  # Imprime el resultado
```

Bloque lógico



Cuando se escribe código en Python haciendo uso de buenas prácticas de programación, aprovechando las fortalezas del lenguaje, se denomina **Código Pythónico.**

Python es **Interpretado**. Esto significa que no será necesario compilar el código cada vez que se realice un cambio y que ese código no sea dependiente del hardware sobre el que se ejecuta, es decir, es un lenguaje **multiplataforma** gracias a que posee su propia **máquina virtual**.

Cuenta con una extensa **librería estándar** que, además, se complementa con multitud de librerías de terceros gracias a las contribuciones de la comunidad Python en todas sus áreas.

Por supuesto, Python tiene puntos débiles. Se considera **lento** en comparación con los lenguajes compilados (P. ej., C), aunque **Python 3** ha hecho mejoras en este aspecto. Aunque existen diferentes soluciones para mejorar esto, una de las más llamativas tiene que ver con la característica de Python de integrarse con otros lenguajes. Así, para mejorar el rendimiento, se ha optado por crear librerías escritas en otros lenguajes compilados de manera que se pueden utilizar en Python de forma transparente para el desarrollador. Es el caso, por ejemplo, de la librería **Pandas**, que permite el uso de operaciones numéricas y científicas trabajando sobre grandes cantidades de datos de forma eficiente y que está desarrollada, en su mayoría, en código **C**.

Hay que apuntar, por último pero no menos importante, que Python es código **libre y gratuito**.

4. Ámbitos de uso.

Al ser un **lenguaje de propósito general**, se utiliza en multitud de ámbitos de la informática.

La siguiente información hace referencia a los ámbitos fundamentales de Python, enumerando sus librerías más utilizadas, pero es importante que tengamos muy en cuenta que cada uno de estos ámbitos es muy extenso y el estudio de cada uno de ellos por separado debe ser más profundo.

4.2. Programación a nivel de sistema operativo.

Python permite interactuar con el sistema operativo mediante la creación de scripts, programas completos o utilidades propias del sistema como lanzamiento de comandos, manejo de ficheros, interacción con sockets del sistema, etc.

Gracias a su portabilidad, estos programas se desarrollan una vez pero se pueden ejecutar en cualquier sistema operativo.

4.3. Aplicaciones con Interfaz de usuario.

Mediante la librería estándar de Python se pueden crear aplicaciones con interacción de usuario tanto por línea de comandos como por interfaz gráfica.

Dos de las librerías destinadas a facilitar hacer CLI (interfaz por línea de comando) son **argparse** y **click.**

Para crear aplicaciones multiplataforma y de escritorio que se integren perfectamente con la apariencia del sistema operativo, la librería básica es **Tkinter.** Existen otras librerías de más alto nivel para estos cometidos como son **PyGTK** y **PyQT** que integran GTK y Qt respectivamente.

4.4. Aplicaciones web e iteracción con servicios web.

Uno de los pilares fundamentales de las aplicaciones que se pueden desarrollar con Python son las aplicaciones web en todos sus ámbitos, desde páginas web estáticas utilizando **Pelican** hasta juegos generados en JavaScript mediante el uso de la librería **Pyjamas** que es capaz de transformar código Python en JavaScript. En los últimos años, el uso de Pyjamas se ha ido reemplazando por **Transcrypt**, **Brython** o **Skulpt**.

Para el desarrollo web existen múltiples frameworks que permiten el desarrollo de aplicaciones tanto de propósito general como específico. Entre los de propósito general podemos destacar **Django**, que es uno de los líderes en el desarrollo web gracias a sus múltiples funcionalidades entre las que destaca su facilidad de uso con bases de datos relacionales (posee su propio ORM (mapeo relacional de objetos). Entre los de uso específico, destacamos **Falcon**, diseñado para poder gestionar muchísimas peticiones por segundo. Otro en este ámbito es **Starlette**, orientado a aplicaciones en tiempo real, soporta una gran cantidad de peticiones concurrentes (decenas de miles por segundo).

4.5. Interacción con servicios de Internet.

Python se integra fácilmente con servicios web de terceros de múltiples formas pero sobre todo mediante la **API.** Un claro ejemplo es la API de servicios de Google que permite conectarse con los usuarios de sus productos como Maps, YouTube… de forma fácil e intuitiva.

Otras formas de interactuar con servicios de Internet son la transferencia de archivos mediante FTP para lo que se puede utilizar la librería **ftplib,** o el envío de emails de manera fácil y sencilla mediante las librerías **smtplib** y **email.**

4.6. Gestión de contenido.

En el más alto nivel se encuentran las aplicaciones para la gestión de contenidos, tanto web como de escritorio.

Sabemos que los **CMS** (sistemas de gestión de contenidos), ayudan a gestionar contenidos como páginas web, blogs, emails, etc. En Python se cuenta con el framework de software libre **Django-CMS** que no solo ayuda a generar contenido web de forma intuitiva y fácil, sino que se construye sobre el framework Django de manera que si necesitamos extender la aplicación resultará más sencillo.

Dentro del entorno de empresas, destacan las **ERP** (sistemas de planificación de recursos empresariales) que son aplicaciones que ayudan en la gestión de empresas en sus diferentes áreas (RRHH, Nóminas, Contabilidad, etc.). Uno de los más populares ERP de software libre escritos en Python es **Odoo**.

4.7. Aplicaciones científicas y manejo de datos.

Hoy en día, Python es un referente en la comunidad científica y en el manejo de grandes cantidades de datos debido a su simplicidad y claridad a la hora de programarlo.

En lo referente al manejo de datos se cuenta con dos librerías principales, **NumPy** y **Pandas**, que permiten manipular grandes cantidades de datos y hacer operaciones masivas sobre los mismos.

NumPy es un paquete orientado a realizar operaciones matemáticas con vectores de N dimensiones de forma altamente eficiente. Se integra con código C++ o Fortran y muchas utilidades relacionadas con el manejo de datos científicos.

Pandas es una librería especializada en operaciones altamente eficientes sobre grandes conjuntos de datos de varias dimensiones permitiendo una manipulación simple.

Para poder realizar pruebas y cálculos con los datos, es importante contar con una herramienta fácil de usar y potente. Aquí encontramos **Jupyter Notebook,** que permite interactuar con los datos que se están analizando, exportar los resultados, compartirlos, evaluar una parte de los cálculos, etc. Esta herramienta dispone de una versión online (Jupyter Notebook online) y al mismo tiempo,

muchos servicios online soportan el formato para mejorar la visualización de documentos, como **GitHub.**

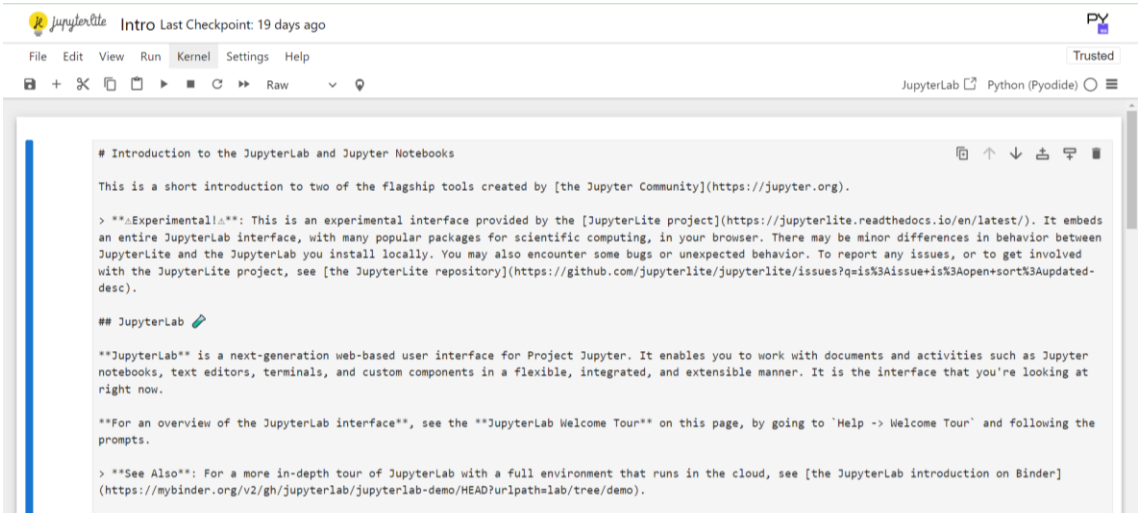


Figura 1. Imagen de Jupyter Notebook

Además de manipular los datos, Python cuenta con librerías para la visualización de esos datos. Una de las más reconocidas es **Matplotlib** que contiene multitud de tipos de gráficos que van, desde los gráficos de líneas hasta los gráficos de puntos o mapas de calor, pasando por gráficos de barras.

Para la representación estadística se usa la librería **Seaborn,** que usa Matplotlib añadiendo visualizaciones orientadas a la estadistica.

Destacamos también la librería **Bokeh,** la cual no sólo permite representar gráficos en dos o tres dimensiones sino que permite interaccionar con ellos.

Por último, debemos reseñar que existen distribuciones de paquetes científicos que unen muchas de las herramientas expuestas en este apartado en un único paquete de software. Un ejemplo claro de este tipo de distribución es **Anaconda** que permite el manejo simple de datos con Jupyter Notebook, el visualizado con Bokeh o Matplotlib o el uso de herramientas de IA como Scikit-learn o TensorFlow.

Anaconda es una distribución completa que contiene todas las herramientas necesarias para la mayoría de ámbitos de la ciencia, la investigación y la ingeniería de datos y se utiliza para instalar todos los componentes de una vez y bajo el mismo entorno.

4.8. Inteligencia artificial y Python

El concepto de IA está en los últimos años muy de moda gracias a las tres partes fundamentales en las que se desarrolla:

- Big Data
- Inteligencia Artificial General
- Deep Learning

Big Data hace referencia al tratamiento de grandes cantidades de datos que en Python se realiza mediante herramientas como NumPy o Pandas vistos anteriormente y mediante el uso de herramientas que nos permiten la obtención de los datos y el almacenamiento eficiente como pueden ser **Apache Spark y Hadoop,** o **Dask** que permite la ejecución paralela del mismo código en máquinas diferentes y si es necesario, escalar esas máquinas de manera que se agiliza la ejecución de los cálculos científicos.

La rama de las ciencias de la computación especializada en **Inteligencia Artificial General** intenta entender la inteligencia humana y desarrollar algoritmos que permitan a las computadoras tener comportamientos similares a los humanos. Los ejemplos más clásicos podrían ser:

- La categorización de imágenes
- Reconocimiento de patrones del habla
- Procesamiento del lenguaje natural
- Robótica

Quizás, en este ámbito cabe destacar la librería de Python **Scikit-learn** que incorpora distintas herramientas muy útiles en este campo.

El aprendizaje profundo (**Deep Learning**) se centra en el desarrollo de algoritmos de aprendizaje automático para analizar datos. Estos datos se expresan de forma matricial o tensorial (N dimensiones) mediante capas de redes neuronales que son especialmente útiles dado que, en la mayoría de los casos, no necesitan la intervención humana para su aprendizaje. En este campo destacamos **TensorFlow,** herramienta de Google ampliamente utilizada, **CNTK** de Microsoft o **Theano.** Estos tres frameworks pueden ser utilizados fácilmente por la librería Python **Keras.**

5. Guía de estilos en Python (PEP-8)



El código de cualquier lenguaje de programación **se lee más veces de las que se escribe**, por lo que es importantísimo tener claro cómo debe escribirse y seguir unas reglas que todos los desarrolladores compartan y acepten.

Python, al igual que el resto de lenguajes, tiene una propuesta en la que se define la Guía de Estilos y que se encuentra perfectamente documentada en el PEP-8 (https://www.python.org/dev/peps/pep-0008/).

- **Indentación**: la indentación en Python es fundamental. Define los bloques lógicos y la lógica entre los mismos; un simple cambio de indentación podría cambiar la lógica del programa.
  La preferencia en la indentación es utilizar espacios (4 espacios) en lugar de tabulación. Cabe destacar que en Python 3, si se utilizan las dos formas de indentación (espacios y tabulación) se producirá un error de sintaxis (aunque muy posiblemente, si utilizas un editor de código o un IDE moderno, lo corrija por defecto y no se produzca el error…).
- **Longitud de las líneas:** la longitud máxima de una línea debe ser de 79 caracteres para el código y 72 para los comentarios y la documentación.
  Aunque sigue existiendo esta recomendación, hoy en día muchos desarrolladores utilizan anchos de línea de 100 o 120 caracteres. Dependiendo de tus preferencias, es posible ajustar esta longitud en la mayoría de los editores mencionados. Cuando la longitud de una línea sea sea mayor que la cantidad máxima predefinida se utilizará el **carácter "\"** para hacer un salto de línea y que continúe el texto en la siguiente.
- **Espacios, saltos de línea y líneas en blanco**: es mejor evitar los espacios y líneas en blanco que parezcan exagerados como en el siguiente ejemplo.

```
3   # Definimos variables
4
5   num1           = 1
6   num2           = 2
7   variable_larga = 3
8
9
10
11  resultado = num1 + num2
```

Los saltos de línea se añaden antes de los operadores lógicos para mejorar la legibilidad del código y siempre procurar que estén alineados al mismo nivel.

```
11   Beneficio = (bruto
12               + interes
13               + alquiler
14               + penalizacion)
```

A la hora de añadir líneas en blanco existen las siguientes reglas:

  - Funciones generales o clases se rodean con dos líneas en blanco
  - Métodos dentro de clases se rodean con una línea.
  - Se pueden usar líneas en blanco de forma moderada entre secciones lógicas
- **Otros consejos generales:** la importación de módulos se hará siempre al comienzo del fichero. El orden será: módulos de librería estándar, módulos de terceros, módulos propios. Además, dentro de cada sección se mantendrá el orden alfabético.
- **Comentarios y documentación:** estas son las reglas que definen cómo deben escribirse los comentarios del código.
  - Es mejor no tener comentarios que tenerlos y que contradigan el código
  - Deben estar compuestos de una o dos frases completas y terminar con punto al final
  - Se pide que los comentarios estén escritos siempre en inglés salvo que estén completamente seguros de que quien leerá ese código entenderá el idioma.
  - Los comentarios de una sola línea se harán de forma moderada, utilizando el carácter "#" detrás de la línea de código que se quiere documentar y siempre que el objetivo de la línea de código no sea obvio en sí mismo.
  - La documentación de código (**docstrings**) se utilizarán siempre los tres caracteres seguidos """ (tres comillas dobles) para abrir y cerrar el bloque de comentarios.
  - Los módulos públicos, funciones, clases y métodos deben tener un docstring.
- **Convención de nombres:** la convención en cuanto a los nombres establece cómo deben nombrarse los identificadores de cada parte del lenguaje. Las más usadas son:

  - CamelCase→ se construye uniendo palabras clave una detrás de otra, utilizando la primera letra de cada palabra en mayúscula.
    VariableNumero

  - snake_case→ se construye uniendo las palabras con una barra baja (_).
    variable_numero

  - SCREAMING_SNAKE_CASE→ se construye de forma similar a snake_case pero con todos los caracteres en mayúsculas.
    VARIABLE_NUMERO

  - Mayúsculas o minúsculas→ son palabras en las que todas las letras son en uno u otro formato.

A continuación definimos cómo se usan estas convenciones de nombres en Python:

- Para definir que un objeto está protegido se añade un carácter "_" como prefijo del nombre.
- Los nombres que terminan con el carácter "_" se utilizan para evitar conflictos con palabras reservadas como class, def, if, etc.
- Cuando un nombre comienza por doble "_", por ejemplo __foo, dentro de cualquier clase, por ejemplo Bar, se trata de manera diferente dado que el compilador lo cambiará internamente por _Bar__foo (en el ejemplo). Es lo que se conoce como Python's mangling rules. Esto se asemeja a hacer un objeto privado y evita colisiones de nombres.
- Para nombrar **paquetes** o **módulos** se utilizarán nombres en minúsculas o snake_case, intentando que sean lo más cortos posible.
- Para los nombres de las **clases** y los **tipos de variables** se usa CamelCase.
- El primer parámetro para los métodos de instancias y para los métodos de clase será self y cls, respectivamente.
- Para los **nombres de las funciones** se utilizará snake_case. Prefijo con un solo "_" si la función es local al módulo y con dos "__" para evitar colisiones con la herencia al hacer uso de las "Python's mangling rules".
- Para las **constantes** se nombrarán mediante SCREAMING_SNAKE_CASE.

Para finalizar, cabe destacar que existen en Python analizadores de código que comprueban si se ha incumplido alguna de las reglas establecidas por la PEP-8. La mayoría de los editores modernos, como es el caso de Visual Studio Code, disponen de extensiones, como **Pylint,** que además de comprobar si se cumple con la sintaxis, alerta de problemas en el código e incluso es capaz de realizar diagramas UML.

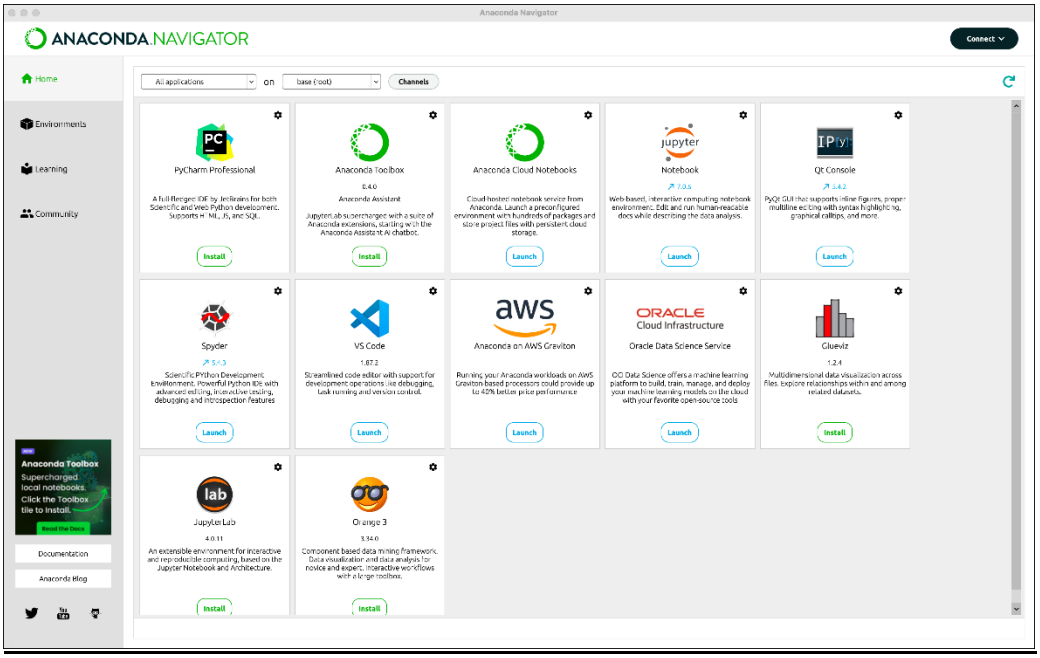
6. Distribuciones Python.

Existen distribuciones Python cuyo objetivo es unir diferentes paquetes de librerías utilizados en un ámbito específico con la finalidad de facilitar la instalación de todos ellos a la vez y no perder tiempo instalando cada componente por separado.

Por lo general, estas distribuciones tienen más herramientas de las que puede utilizar o necesitar un principiante o incluso un experto. A continuación se muestran algunas de las más conocidas.

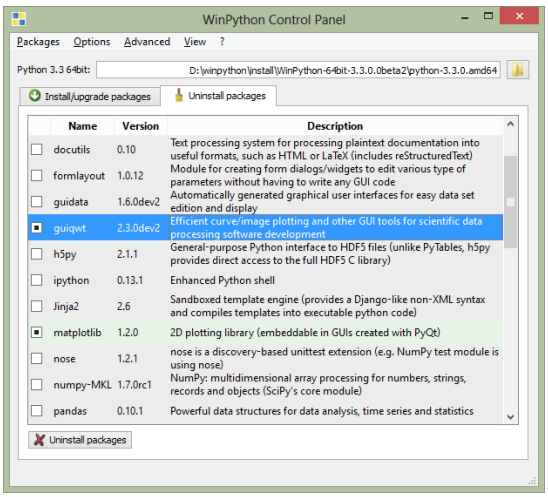
- **Anaconda:** da soporte a un amplio abanico de profesionales de la industria del software y particularmente al sector científico y matemático

que usa Python. Es, seguramente, la distribución más conocida y utilizada para el análisis de datos y ciencia.
Los paquetes más destacables que tiene son Pandas, NumPy, Scikit-learn, Numba, Matplotlib, Bokeh, soporte de Ipython y Jupyter Notebook, entre otros. Dispone de un gestor de paquetes llamado **Conda** que permite la instalación de más de 1500 paquetes científicos de **Python** y **R.** Este gestor viene integrado en Anaconda pero no es dependiente de él de manera que es posible utilizar el gestor de paquetes para diferentes lenguajes como Ruby, R, Lua, Java, JavaScript y C++ entre otros sin necesidad de instalar la distribución de Anaconda.



La imagen superior es la interfaz gráfica llamada Anaconda Navigator que gestiona los paquetes instalados y las aplicaciones.

- **WinPython:** es un proyecto de distribución de Python de software libre. Da soporte a la instalación de Python en un sistema operativo Windows con un programa portable y autocontenido, sin necesidad de instalar por separado el intérprete ni ningún paquete fuera del ecosistema del programa.



Además del intérprete de Python dispone de distintos programas y librerías como los editores **Spyder** y **Pyzo,** herramientas de desarrollo de aplicaciones con **Qt** o librerías preinstaladas como **Pandas**, **SciPy**, **Jupyter**, entre otras.

- **Enthought Canopy:** es una distribución orientada al ámbito científico aunque también permite programar aplicaciones de escritorio. Cuenta con un gestor de paquetes propio llamado **EDM.** Cuenta con una interfaz gráfica a modo de editor pero, dado que la están eliminando, recomiendan usar otro como por ejemplo VSC.
- **ActivePython:** a diferencia de Anaconda o Canopy, intenta ser más trasversal y no enfocarse sólo en el desarrollo matemático, estadístico o de inteligencia artificial sino también en otros sectores, como el desarrollo de aplicaciones web, la seguridad o el testeo de aplicaciones. Tiene soporte para Intel Math Kernel Library que es una librería que ayuda a acelerar el procesamiento matemático.

7. El intérprete de Python

Python es un lenguaje interpretado, lo que significa que el código fuente no necesita ser compilado a código máquina específico del hardware donde se ejecuta sino que se ejecuta directamente sobre cualquier sistema con la máquina virtual de Python instalada. Lo que hace realmente el intérprete de Python es traducir el código a bytecode (código intermedio) y ejecutarlo.

Cualquier instalación de Python tiene como mínimo dos componentes, el intérprete y la librería estándar (que contiene los módulos, funciones, constantes, tipos de datos…) Dependiendo de la implementación de Python, el intérprete puede estar escrito en C, Java, .Net, etc.

Se podría definir el **intérprete** como un programa que se encarga de ejecutar otros programas.

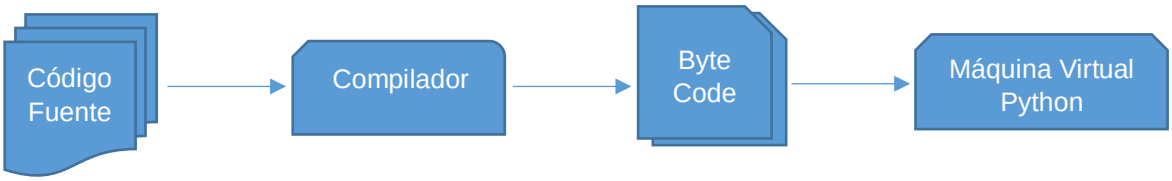
7.1. Estructura del intérprete.

Por una parte tenemos el código fuente, compuesto de ficheros de texto plano con una gramática específica (lenguaje Python) y con extensión ".py"

Por otra parte tenemos los ficheros **byte code**, que son el resultado de una compilación rápida que se efectúa justo antes de comenzar la ejecución. Ese código byte code se puede ejecutar en cualquier máquina virtual Python.

Por último tenemos la PVM (máquina virtual de Python) que se encarga de ejecutar los ficheros byte code. Esta máquina es la única parte dependiente del hardware utilizado y debe ser compilada cuando se instala Python por primera vez en el sistema.

Lo que se denomina **intérprete** de Python es el programa completo que analiza el código fuente, genera los ficheros compilados y ejecuta el código usando la máquina virtual.



Los ficheros que contienen el byte code tienen extensión ".pyc", y no son necesarios para la ejecución dado que, si por el motivo que sea no pueden generarse, el byte code será generado e insertado directamente en memoria.

8. Implementaciones de Python

Normalmente cuando hablamos de la implementación de Python se hace referencia a la que usa C, denominada CPython, pero el intérprete puede ser implementado en otros lenguajes. La diferencia principal entre unas y otras es la habilidad para utilizar librerías escritas en un lenguaje específico.

Veamos las implementaciones más conocidas:

- **CPython**: es la implementación original. Está hecha en ANSI C y es la más utilizada. Si se pretende crear código que sea lo más compatible o software libre y no se tiene una restricción o dependencia fuerte con algún otro lenguaje (como Java o C#) es la mejor opción.
- **Jython:** es una implementación sobre la máquina virtual de Java y se puede conocer también como Jpython. Puede ejecutarse junto con código Java. La diferencia es que el código fuente Python es transformado de manera transparente a byte codes de Java y ejecutado como cualquier otro programa Java. Como es de suponer, esto permite la interoperabilidad completa entre Python y Java de manera que es posible importar librerías de Java como si fuesen librerías de Python.
- **PyPy**: es una implementación de Python en C que incorpora JIT (just in time compiler), que es un compilador en tiempo de ejecución manteniendo el carácter de interpretado, de manera que lo dota de mayor velocidad de ejecución. Se debe tener en cuenta que no es compatible con todos los programas escritos en Python por lo que hay que valorar muy bien su usabilidad si se utilizar en un proyecto buscando velocidad de ejecución.
- **IronPython**: es una implementación de **C#** que se integra perfectamente en el framework .**NET**

8.1. Desarrollar programas en Python.

Sólo necesitamos un simple editor de textos y un intérprete para poder generar una aplicación Python. Pero como ocurre con todos los lenguajes de programación, existen programas que consiguen que esta tarea sea más fácil e intuitiva. Es aquí donde nos encontramos con los Entornos de Desarrollo Integrados (IDE) que son programas que proveen de herramientas específicas para diversas partes del desarrollo, facilitando la tarea de los desarrolladores al disponer de todo lo que necesitan dentro de un mismo entorno. Así, podemos encontrar:

- **Entornos de desarrollo Open Source.**
  - **Vim→** es un editor de texto orientado a consola. Cuenta con varios modos de operación. Requiere mucho tiempo de aprendizaje y configuración pero a favor tiene que es altamente configurable y modulable. Se organiza en "buffers" que puede dividir de forma vertical u horizontal por lo que podemos tener múltiples ficheros abiertos en la misma pantalla y trabajar con ellos. Existen implementaciones con interfaz de ventana. Viene instalado por defecto en la mayoría de distribuciones Unix.
  - **Atom→** es un editor de texto moderno, multiplataforma. Creado por los mismos desarrolladores de GitHub. Es intuitivo y fácil de usar. Tiene soporte para Python por lo que detecta y colorea el código Python. Permite instalar nuevos plugins, tiene navegador de archivos, capacidad de utilizar múltiples paneles de separación horizontal y verticalmente e integración por defecto con Git y GitHub
  - **IDLE→** se define como "entorno de aprendizaje y desarrollo integrado". Escrito totalmente en Python mediante las librerías de Tkinter, permite edición de ficheros, depuración de código, utilizar varias ventanas de edición, coloreado de gramática Python, autocompletado, entre otras muchas posibilidades.
  - **PyDev→** es un IDE que se integra en **Eclipse** para desarrollar con Python. Tiene autocompletado de código, ayudas para acceder a la definición de los objetos, depurador, integración de Pylint y muchas otras características importantes. Una ventaja es que una vez instalado dentro de Eclipse, todas las herramientas necesarias para programar en Python ya están instaladas.
  - **Spyder→** IDE orientado al desarrollo científico utilizando Python. Está presente en el paquete de Anaconda por defecto. Incorpora NumPy, Pandas para el manejo de datos, Matplotlib para la representación de imágenes…Se puede utilizar por separado de Anaconda. Es completo y útil incluso para desarrollos no científicos.
  - **Visual Studio Code→** IDE de código libre, ligero y rápido, estable y multiplataforma. Permite el desarrollo de muchísimos lenguajes

incluidos C#, C++, Java, PHP o Python. La utilización de cualquiera de estos lenguajes se consigue mediante la instalación de "Extensiones". Incorpora muchísimas características que hacen que sea uno de los editores (en sus comienzos era más un editor que un IDE) más utilizados por los desarrolladores. Se integra perfectamente con Git y GitHub y tiene soporte para Jupyter Notebook.

- **Entornos de desarrollo de código cerrado.**

  Estos IDE son aquellos que están desarrollados por compañías privadas que establecen licencias de pago sobre el producto.

  - **Sublime Text** → editor de texto orientado a multitud de lenguajes con muchas características interesantes para el desarrollo de aplicaciones. Además de distintas funcionalidades que hacen que sea un editor muy utilizado por la comunidad de desarrolladores, cuenta con un sistema de paquetes para expandir las funcionalidades. Tiene soporte para Python mediante el paquete Anaconda.
  - **PyCharm**→ es considerado uno de los mejores IDE para el desarrollo de Python. Está disponible en dos versiones, la Community Edition y la Professional. La versión Community es muy potente y cuenta con un sinfín de funcionalidades que lo convierten en uno de los líderes indiscutibles en el desarrollo de cualquier aplicación Python. La versión Professional va un paso más allá, complementando a la anterior con algunas nuevas funcionalidades que mejoran la productividad y permiten que prácticamente no sea necesario salir del IDE para utilizar alguna herramienta externa.

Cada desarrollador debe elegir el IDE que más se ajuste a sus necesidades de desarrollo. Un listado ordenado desde los más comunes hasta los más específicos para Python sería: Vim, Emacs, Sublime Text, Atom, PyDev, Visual Studio Code, Spyder, ActivePython, PyCharm.



Para este módulo recomendamos utilizar Visual Studio Code principalmente por su sencillez, potencia, escalabilidad y porque actualmente es uno de los IDE más utilizados en el entorno profesional.

9. Primeros programas en Python con VSC

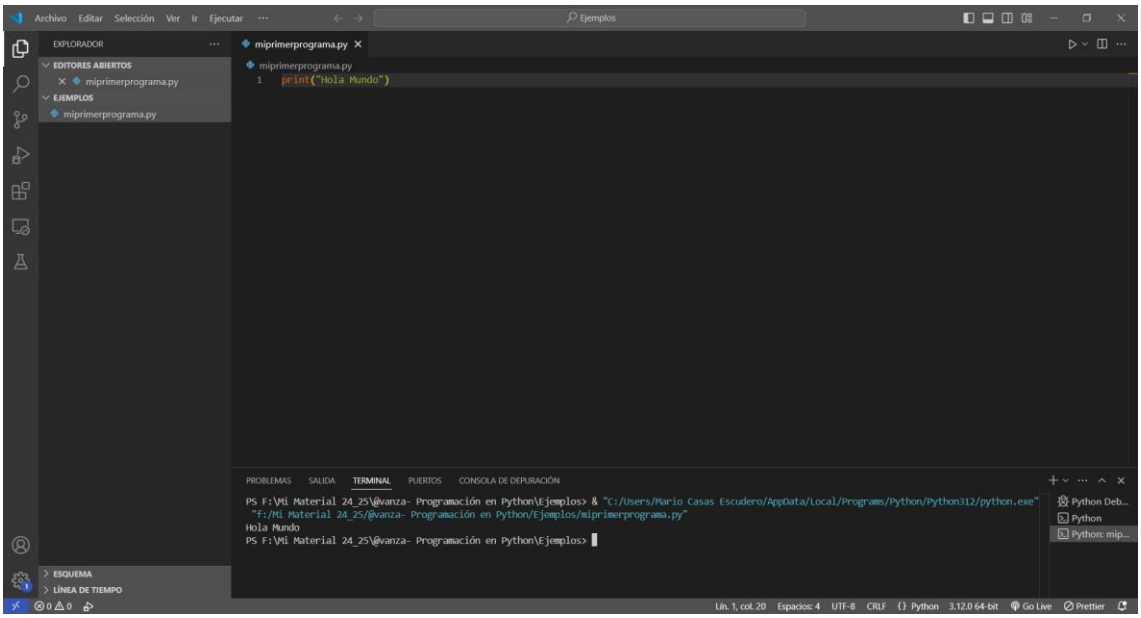
En este punto, debes tener instalado el intérprete de Python y el IDE Visual Studio Code con las extensiones que hacen posible el desarrollo con Python.

En la sección de recursos, si no lo has hecho ya, tienes una guía para llevar a cabo la instalación.

Pues bien, existe una práctica comúnmente conocida a modo de iniciación para cualquier lenguaje de programación que consiste en emitir un mensaje al mundo con la frase “Hola Mundo”.

Mientras en otros lenguajes es necesario crear más de tres instrucciones para llevar a cabo esto, en Python es tan sencillo como ejecutar la siguiente instrucción:

```
print(“Hola Mundo”)
```



He querido mostrar en este primer “ejemplo” todo el área de trabajo de VSC para que veas cómo iremos trabajando. En el panel izquierdo tenemos el área de exploración de nuestros ficheros, donde crearemos nuestros proyectos. En este caso, utilizo una carpeta de EJEMPLOS y ahí dentro iré creando los distintos ejemplos de código que iremos realizando. En este primer programa he creado el fichero **miprimerprograma.py**

En el momento que le damos al fichero la extensión “.py” VSC detecta que se trata de código Python y los edita como tal.

Escribimos la instrucción comentada en el área del editor de código, salvamos el fichero pulsando Ctrl+S o mediante la opción de menú Archivo->Guardar y finalmente ejecutamos. Obtenemos el mensaje en el terminal, dado que la función utilizada “**print(msg)**” lo que hace precisamente es eso, muestra por terminal el mensaje pasado como parámetro.